Deep Reinforcement Learning for Edge Caching and Content Delivery in Internet of Vehicles

Yueyue Dai *, Du Xu *,Yunlong Lu †, Sabita Maharjan‡§ and Yan Zhang §
*School of Information&Communication Engineering, University of Electronic Science and Technology of China, China
†Institute of Network Technology, Beijing University of Posts and Telecommunications, China
‡ Simula Metropolitan Center for Digital Engineering, Norway
§ Department of Informatics, University of Oslo, Norway

***Abstract*—To enable the emerging vehicular applications and multimedia services in an Internet of Vehicles (IoV) framework, edge caching is a promising paradigm which can cache content in proximity to vehicles, and thus alleviate heavy load on backhaul links and contribute in reducing transmission latency. However, in a multi-access vehicular network, complex content delivery and high mobility of vehicles introduce new challenges to support edge caching in a dynamic environment. Deep Reinforcement Learning (DRL) is an emerging technique to solve the issue with time-varying feature. In this paper, we utilize DRL to design an optimal vehicular edge caching and content delivery strategy for minimizing content delivery latency. We first propose a multi-access edge caching and content delivery framework in vehicular networks. Then, we formulate the vehicular edge caching and content delivery problem and propose a novel DRL algorithm to solve it. Numerical results demonstrate the effectiveness of proposed DRL-based algorithm, compared to two benchmark solutions.**

I. Introduction

The advancements in Internet of Vehicles (IoV) and wireless technologies have paved a path towards intelligent transportation and enhanced driving experience by utilizing new vehicular applications and multimedia services, such as automated driving, smart navigation and on-vehicle videos [1], [2], [3]. With rapid increase in the use of such applications and services, the content requirements are soaring at a tremendous rate [4], [5]. However, long distance between vehicles and the cloud, and the limited backhaul link capacity pose significant challenges for supporting massive content delivery while also fulfilling the low-latency requirement in vehicular networks.

Edge caching is a promising approach to alleviate the need of large backhaul bandwidth and reduce content delivery delay by providing storage capability at wireless access networks [4], [6]. By caching popular content close to users, the requested content can be delivered from near-by edge nodes without long-distance communication. There has been considerable amount of work focusing on edge caching and content delivery [7], [8], [9]. The authors in [7] investigated the femto-caching and content placement problem to minimize the total expected content delivery delay based on a given popularity distribution. In [8], the authors proposed a random content caching approach with device-to-device communications. The authors in [9] formulated the caching placement and content delivery as two optimization problems which aimed to maximize cache hit ratio and minimize energy consumption, respectively. The aforementioned studies mainly explore the caching placement and content delivery policies in cellular networks. However, these typical edge caching techniques cannot be directly applied to multi-access vehicular networks which involve communications between vehicles, vehicles and network infrastructures (i.e., Macro-Base Stations (MBSs) and Road-Side Units (RSUs)). In addition, high mobility of vehicles leads to dynamic network topology and possibly less stable wireless channel condition, thus making edge caching and content delivery strategy design more challenging.

Deep Reinforcement Learning (DRL) has recently attracted much attention in edge caching and content delivery. The authors in [10] utilized DRL to investigate a secure and intelligent device-to-device content caching and resource allocation strategy for maximizing system utility in vehicular networks. In [11], the authors estimated the unknown popularity of cached contents by using multi-armed bandit learning, and then proposed a content caching policy based on the learned content popularity distribution in wireless networks. Utilizing actor-critic reinforcement learning, the authors in [12] proposed a scheme to solve the joint content caching, computation offloading, and resource allocation problems in fog-enabled Internet of Things (IoT) networks.

Leveraging DRL approach, we propose a multi-access edge caching and content delivery strategy in order to carefully address the following critical challenges: 1) How to efficiently deploy edge caching on RSUs and vehicles, 2) How to design a content delivery strategy and allocate resources automatically in multi-access vehicular networks. Taking vehicular mobility into consideration, we exploit DRL to design the optimal multi-access edge caching and content delivery strategy for minimizing content delivery latency. The key contributions of our work are as follows:

- We propose a multi-access vehicular edge caching and content delivery framework where both RSUs and vehicles can store popular contents, such that they can be delivered to the requesting vehicles from their adjacent vehicles, RSUs or the MBS.
- We utilize the form of DRL to formulate vehicular edge caching and content delivery problem for minimizing the total content delivery latency.
- We propose an efficient vehicular edge caching and

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 16:18:15 UTC from IEEE Xplore. Restrictions apply.

content delivery algorithm to find the solution of the optimization problem based on a novel DRL approach.

The rest of this paper is organized as follows. In Section II, we introduce the system model. We formulate vehicular edge caching and content delivery problem and propose the DRL-based algorithm to solve this problem in Section III and Section IV, respectively. We evaluate the performance of the proposed DRL-based algorithm and provide illustrative results in Section V. We conclude the paper in Section VI.

II. System Model

We consider a multi-access vehicular network consisting of one MBS and M RSUs, as shown in Fig. 1. The set of MBS and RSUs is denoted as $\mathcal{M} = \{0, 1, ..., M\}$, where 0 is the index of the MBS. All M RSUs are connected to the MBS via wired links [13]. There are V vehicles randomly distributed along the road, denoted as $\mathcal{V} = \{1, ..., V\}$. Due to the coexistence of MBS, RSUs, and vehicles, the communication in this network has three types: Vehicle-to-Vehicle (V2V), Vehicle-to-RSU (V2R), and Vehicle-to-MBS (V2I). For ease of description, V2I is defined as the communication between vehicles and the MBS in this paper. Let $\mathcal{F} = \{1, ..., F\}$ denote the set of F contents in the content library. Each content is identified by three features, which can be described as $f = \{s_f, \tau_f, \rho_f\}$. s_f and τ_f denote the size and the maximal content delivery latency of content f, respectively. ρ_f is the popularity of content f. We assume contents are sorted in descending order of popularity. That is, $\rho_1 \geq \rho_2 \geq ... \geq \rho_F$. All contents are considered to be cached in the MBS, that has infinite cache capacity. The RSUs are endowed with edge caching resources such that some popular contents can be cached in the RSUs to reduce content delivery latency. Since vehicles are equipped with a certain amount of caching resource, they can be utilized as mobile caching servers for edge caching. When a vehicle requests a specific content, the requested content may not only be cached on the MBS but also on the RSUs and vehicles such that vehicles can obtain their requested contents locally or from their adjacent vehicles, RSUs, and the MBS.

A. Caching Model

We consider that the popularity of contents follows the Zipf distribution [9]. For content f, the probability that it is requested by vehicles can be denoted as

$$P_f = \frac{{\rho_f}^{-\varepsilon}}{\sum_{n=1}^{F} {\rho_n}^{-\varepsilon}}, \tag{1}$$

where ε is the content request coefficient that controls the relative popularity of contents. A larger ε indicates a higher content reuse, i.e., the first few popular contents account for the majority of requests from vehicles [11].

Each vehicle and RSU have a local cache with capacity of C_i^{veh} ($i \in \mathcal{V}$) and C_j^{rsu} ($j \in \mathcal{M}/\{0\}$), respectively. We define $x_{if}^{vec} \in \{0, 1\}$ as the content caching variable of vehicle i. If content f is cached locally, $x_{if}^{vec} = 1$. Otherwise, $x_{if}^{vec} = 0$. Similarly, $x_{jf}^{rsu} \in \{0, 1\}$ is defined as the content caching variable of RSU j. Due to limited available caching capacity, the total cache size of the contents on the vehicles and RSUs should not exceed their caching capacities.

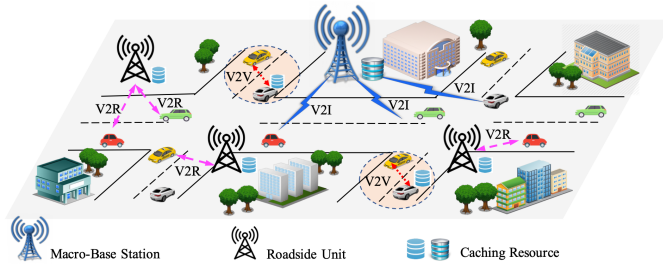

Fig. 1: Illustration of a multi-access vehicular network with edge caching resource.

B. Content Delivery Model

When a vehicle requests a content, it will first check its own cache. If the requested content is cached locally, the vehicle can directly obtain the requested content without any transmission. Otherwise, the vehicle can obtain the requested content mainly in the following three ways:

1) *V2V content delivery:* A V2V link can only be established between two vehicles within distance γ. The set of potential neighboring vehicles of vehicle i can be denoted as $\mathcal{V}_i = \{k \in \mathcal{V}/\{i\} | d_{ik} \leqslant \gamma\}$, where d_{ik} is the distance between vehicle i and vehicle k. The achievable data rate from vehicle k to vehicle i can be expressed as

$$R_{k \to i} = B_v \log_2(1 + \frac{p_k h_{ik} d_{ik}^{-\alpha}}{\sigma^2}), \tag{2}$$

where B_v is the channel bandwidth, p_k is the transmission power of vehicle k, h_{ik} is the channel gain between vehicle i and vehicle k, α denotes the path loss exponent, and σ^2 denotes the noise power.

If vehicle k caches the requested content, vehicle i can obtain the requested content directly from vehicle k. We define $y_{ikf}^{v2v} \in [0, 1]$ as the content delivery variable, which represents the ratio of the requested content to the total content that vehicle i can obtain from vehicle k. The corresponding V2V content delivery time is $y_{ikf}^{v2v} s_f / R_{k \to i}$.

2) *V2R content delivery:* Vehicles under the same RSU utilize the same frequency band, Thus, the inter-cell interference among V2R links cannot be ignored. Let B_j^s denote the channel bandwidth of RSU j. The achievable transmission rate from RSU j to vehicle i can be expressed as:

$$R_{j \to i} = b_{ij}^s log(1 + \frac{p_j h_{ij} d_{ij}^{-\alpha}}{\sigma^2 + \sum_{i'=1, i' \neq i}^{V} p_j h_{i'j} d_{i'j}^{-\alpha}}), \tag{3}$$

where p_j is the transmission power of RSU j, b_{ij}^s is the channel bandwidth that RSU j allocates to vehicle i, h_{ij} and d_{ij} are the channel gain and the distance between RSU j and vehicle i. $\sum_{i'=1, i' \neq i}^{V} p_j h_{i'j} d_{i'j}^{-\alpha}$ is the downlink inter-cell interference among V2R links.

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 16:18:15 UTC from IEEE Xplore. Restrictions apply.

We define $y_{ijf}^{v2r} \in [0,1]$ as the content delivery variable, which represents the ratio of the requested content to the total content that vehicle i can obtain from RSU j. The V2R content delivery time will be $y_{ijf}^{v2r} s_f / R_{j\to i}$.

3) *V2I content delivery:* We define transmission between the MBS and vehicles as V2I transmission, as shown in Fig. 1. The Orthogonal Frequency Division Multiple Access (OFDMA) is employed for wireless transmission between the MBS and vehicles. The channel bandwidth of the MBS is B_m, which can be divided into W orthogonal sub-carriers. Each vehicle associated to the MBS is allocated one orthogonal sub-carrier. The achievable transmission rate from the MBS to vehicle i is:

$$R_{0\to i} = \frac{B_m}{W} log(1 + \frac{p_m h_{i0} d_{i0}^{-\alpha}}{\sigma^2}), \tag{4}$$

where p_m is the transmission power of the MBS. h_{i0} and d_{i0} are the channel gain and the distance between the MBS and vehicle i, respectively.

If the requested content cannot be totally obtained from the above mentioned methods, the rest will be delivered from the MBS(i.e., $y_{i0f}^{v2i} = 1 - \sum_{k=1}^{V} y_{ikf}^{v2v} - \sum_{j=1}^{M} y_{ijf}^{v2r}$). The content delivery time via V2I transmission is $y_{i0f}^{v2i} s_f / R_{0\to i}$.

C. Vehicle Mobility Model

The Manhattan grid model is adopted as the vehicle mobility model in this paper [14]. Let η_i denote the driving direction of vehicle i, where $\eta_i \in \{north, south, west, east\}$. According to [14], the probability that vehicle i moves at an intersection will be

$$P_{\eta_i} = \frac{\frac{1}{\delta_{int}\nu_i}}{\frac{1}{\delta_{int}\nu_i} + \frac{T_i^{wait} P_i^{wait}}{2}} = \frac{2}{2 + T_i^{wait} P_i^{wait} \delta_{int} \nu_i}, \tag{5}$$

where δ_{int} is the density of intersections, ν_i denotes the velocity of vehicle i, T_i^{wait} denotes the maximum tolerant waiting time of vehicle i at the intersection, P_i^{wait} is the probability that vehicle i has to wait. In (5), $\frac{1}{\delta_{int}\nu_i}$ represents the time that vehicle i travels from one intersection to another and $\frac{T_i^{wait} P_i^{wait}}{2}$ denotes the stopping time of vehicle i.

Further, vehicle i may stop at an intersection, which can be denoted as ζ_i. The probability that a vehicle stops at an intersection can be expressed as

$$P_{\zeta_i} = 1 - P_{\eta_i} = \frac{T_i^{wait} P_i^{wait} \delta_{int} \nu_i}{2 + T_i^{wait} P_i^{wait} \delta_{int} \nu_i}. \tag{6}$$

III. Problem Formulation

In this section, we formulate the edge caching and content delivery problem as Markov Decision Process (MDP) to minimize the total content delivery delay with system state, action, and reward.

A. System State

At each time step, the DRL agent observes network and collects the following components to constitute system state. Let $\mathcal{S}$ denote the system state space, the state $s_t \in \mathcal{S}$ at time slot t can be defined as

$$s_t = \{[V_i(t)], \mathbf{R}(t), \mathbf{F}(t), [\eta_i(t)]\} \tag{7}$$

where

- $V_i(t)$: the set of neighboring vehicles of vehicle i at time slot t;
- $\mathbf{R}(t) = \{[R_{k\to i}(t)], [R_{j\to i}(t)], [R_{0\to i}(t)]\}$: the set that represents data rate of V2V, V2R, and V2I links for content delivery at time slot t;
- $\mathbf{F}(t) = \{[x_{if}^{vec}], [x_{jf}^{rsu}]\}$: the set of cached contents on vehicles and RSUs at time slot t;
- $\eta_i(t)$: the driving direction of vehicle i at time slot t;

Note that because of mobility, the location of each vehicle is time-varying such that the data rates of V2V, V2R and V2I links are also time-varying.

B. System Action

The action of vehicular edge caching and content delivery consists of two phases: content placement phase and content delivery phase. In the content placement phase, RSUs and vehicles independently cache some popular contents based on their caching capacities and content popularity distribution. In the content delivery phase, a joint content delivery and bandwidth allocation problem is triggered to minimize the average content delivery latency. Let $\mathcal{A}$ denote the action space, the action $a_t \in \mathcal{A}$ at time slot t is defined as

$$a_t = \{[x_{if}^{vec}(t)], [x_{jf}^{rsu}(t)], [y_{ikf}^{v2v}(t)], [y_{ijf}^{v2r}(t)], [y_{i0f}^{v2i}(t)], [b_{ij}^{s}(t)]\} \tag{8}$$

where

- $x_{if}^{vec}(t)$: represents the decision whether to cache content f on vehicle i or not. Note that $\sum_{f=1}^{F} x_{if}^{vec}(t) s_f \le C_i^{veh}$,
- $x_{jf}^{rsu}(t)$: represents the decision whether to cache content f on RSU j or not. Note that $\sum_{f=1}^{F} x_{jf}^{rsu}(t) s_f \le C_j^{rsu}$,
- $y_{ikf}^{v2v}(t)$: represents the ratio of the requested content that vehicle i can obtain from vehicle k,
- $y_{ijf}^{v2r}(t)$: represents the ratio of the requested content that vehicle i can obtain from RSU j,
- $y_{i0f}^{v2i}(t)$: represents the ratio of the requested content that vehicle i can obtain from the MBS,
- $b_{ij}^{s}(t)$: represents the bandwidth RSU j allocates to vehicle i.

C. Reward Function

After an action a_t is taken, the system will receive an immediate reward $\Upsilon(s_t, a_t)$. The objective of edge caching and content delivery problem is to minimize the total content delivery delay. Thus, the immediate reward can be written as

$$\Upsilon(s_t, a_t) = -\mathbb{E}\left[\sum_{i=1}^{V}\sum_{f=1}^{F} P_f T_{if}(t)\right]. \tag{9}$$

where $T_{if}(t)$ denotes the content delivery delay for vehicle i requesting content f at time slot t. Since vehicle i can obtain the requested content from its own cache, adjacent vehicle,

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 16:18:15 UTC from IEEE Xplore. Restrictions apply.

RSU, and the MBS, the content delivery delay $T_{if}(t)$ can be expressed as

$$T_{if}(t) = \underbrace{x_{if}^{vec}(t) y_{iif}^{vec}(t) T_{i\to i}(t)}_{local\ cache} + \underbrace{\sum_{k\in\mathcal{V}_i} \frac{x_{kf}^{vec}(t) y_{ikf}^{vec}(t) s_f}{R_{k\to i}(t)}}_{V2V\ delivery} + \underbrace{\sum_{j\in\mathcal{M}/\{0\}} \frac{x_{jf}^{rsu}(t) y_{ijf}^{rsu}(t) s_f}{R_{j\to i}(t)}}_{V2R\ delivery} + \underbrace{\frac{y_{i0f}^{v2i}(t) s_f}{R_{0\to i}(t)}}_{V2I\ delivery} \tag{10}$$

where $T_{i\to i}(t) = 0$ and $y_{iif}^{vec}(t) + \sum_{k\in\mathcal{V}_i} y_{ikf}^{vec}(t) + \sum_{j\in\mathcal{M}/\{0\}} y_{ijf}^{rsu}(t) + y_{i0f}^{v2i}(t) = 1$.

If the requested content cannot be obtained within its maximal content delivery latency τ_f, the DRL agent will receive a penalty by executing current action (i.e., $\Upsilon(s_t, a_t) = penalty$, where penalty is a negative constant).

The optimal vehicular edge caching and content delivery strategy is to maximize the long-term revenue of the vehicular networks, and the cumulative total reward is given by

$$Reward = \max \mathbb{E}\left[\sum_{t=0}^{T-1} \epsilon^t \Upsilon(s_t, a_t)\right] \tag{11}$$

where $\epsilon \in [0, 1]$ is the discounted factor.

IV. DRL-based Vehicular Edge Caching and Content Delivery Scheme

We exploit the advanced deep reinforcement learning algorithm, Deep Deterministic Policy Gradient (DDPG), to solve vehicular edge caching and content delivery problem. The DDPG agent is composed of three modules: primary network, target network, and replay memory. Primary network explores the detailed edge caching and content delivery policy by mapping current state to an action. Primary network consists of two deep neural networks, namely primary actor neural network and primary critic neural network. Target network is used to generate the target value for training the primary critic neural network. The structure of target network is the same as the structure of primary network but with different parameters. Replay memory is used to store experience tuples. Experience tuples include current state, the selected action, reward, and next state, which can be randomly sampled for training primary network and target network. The details of primary actor neural network, primary critic neural network, and target network are described as follows:

1) *The primary actor neural network:* The primary actor neural network explores the detailed edge caching and content delivery policy. The explored policy can be defined as a function parametrized by θ^μ, mapping current state to an action

$$\mu(s_t|\theta^\mu) = \hat{a}, \tag{12}$$

where $\hat{a}$ is a proto-actor action generated by the mapping.

Based on the explored policy, the primary actor neural network constructs an action as follows [15]

$$a_t = \mu(s_t|\theta^\mu) + \mathfrak{N}_t, \tag{13}$$

where $\mu(s_t|\theta^\mu)$ is the explored edge caching and content delivery policy and $\mathfrak{N}_t$ is the Ornstein-Uhlenbeck noise.

The primary actor neural network updates the network parameter θ^μ using the sampled policy gradient, which is given as

$$\bigtriangledown_{\theta^\mu} J \approx \mathbb{E}\left[\bigtriangledown_a Q(s, a|\theta^Q)|_{s=s_t, a=\mu(s_t)} \bigtriangledown_{\theta^\mu} \mu(s|\theta^\mu)|_{s=s_t}\right] \tag{14}$$

where $Q(s, a|\theta^Q)$ is an action-value function and will be introduced in the following.

2) *The primary critic neural network:* The primary critic neural network evaluates the performance of the selected action based on the action-value function. The action-value function is calculated by the Bellman optimality equation and can be expressed as

$$Q(s_t, a_t|\theta^Q) = \mathbb{E}\left[\Upsilon(s_t, a_t) + \epsilon Q(s_{t+1}, \mu(s_{t+1})|\theta^Q)\right], \tag{15}$$

Here, the primary critic neural network takes both current state s_t and next state s_{t+1} as input to calculated $Q(s_t, a_t|\theta^Q)$ for each action.

The primary critic neural network updates the network parameter θ^Q by minimizing the loss function $Ls(\theta^Q)$. The loss function is defined as

$$Ls(\theta^Q) = \mathbb{E}\left[(y_t - Q(s_t, a_t|\theta^Q))^2\right] \tag{16}$$

where y_t is the target value and can be obtained by

$$y_t = \Upsilon(s_t, a_t) + \epsilon Q'(s_{t+1}, \mu'(s_{t+1}|\theta^{\mu'})|\theta^Q). \tag{17}$$

3) *The target network:* The target network can be regarded as an old version of the primary network with different parameters $\theta^{\mu'}$ and $\theta^{Q'}$. At each iteration, the parameters $\theta^{\mu'}$ and $\theta^{Q'}$ is updated based on the following definition:

$$\begin{aligned} \theta^{\mu'} &= \omega\theta^\mu + (1-\omega)\theta^{\mu'} \\ \theta^{Q'} &= \omega\theta^Q + (1-\omega)\theta^{Q'} \end{aligned} \tag{18}$$

where $\omega \in [0, 1]$.

A. DDPG-based Vehicular Edge Caching and Content Delivery Algorithm

The proposed DDPG-based vehicular edge caching and content delivery algorithm is given in Algorithm 1. Specifically, DDPG agent initializes the edge caching and content delivery policy $\mu(s|\theta^\mu)$ of the primary actor neural network with parameter θ^μ, and initializes the action-value faction $Q(s, a|\theta^Q)$ of the primary critic neural network with parameter θ^Q. Both parameters $\theta^{\mu'}$ and $\theta^{Q'}$ of the target network are also initialized. Then, according to the current policy $\mu(s|\theta^\mu)$ and state s_t, the primary actor neural network generates action a_t at time step t based on (13). According to the observed reward $\Upsilon(s_t, a_t)$ and next state s_{t+1}, DDPG agent constructs a tuple $< s_t, a_t, \Upsilon(s_t, a_t), s_{t+1} >$ and stores it into replay buffer $\mathcal{D}$. Based on mini-batch technique, DDPG agent updates the primary critic neural network by minimizing loss function $Ls(\theta^Q)$ and updates the primary actor neural network using the sampled policy gradient. After a period of training,

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 16:18:15 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 DDPG-based vehicular edge caching and content delivery algorithm

Initialize the primary actor neural network $\mu(s|\theta^{\mu})$ and the primary critic neural network $Q(s,a|\theta^{Q})$ with parameters θ^{μ} and θ^{Q};
Initialize the target actor neural network $\mu'(s|\theta^{\mu})$ and the target critic neural network $Q'(s,a|\theta^{Q})$ with parameters $\theta^{\mu'} \leftarrow \theta^{\mu}$ and $\theta^{Q'} \leftarrow \theta^{Q}$; Initialize replay buffer $\mathcal{D}$;
for each episode **do**
    Initialize the environment setup;
    for each time step t **do**
        Execute action a_t based on (13).
        Observe reward $\Upsilon(s_t, a_t)$ and state s_{t+1} based on (9) and (10);
        Store the tuple $< s_t, a_t, \Upsilon(s_t, a_t), s_{t+1} >$ into replay buffer $\mathcal{D}$;
        Sample a mini-batch of N tuples from replay buffer $\mathcal{D}$;
        Compute the target value y_t based on (17);
        Update the primary critic neural network $Q(s,a|\theta^{Q})$ by minimizing the loss function (16);
        Update the primary actor neural network using the sampled policy gradient (14);
        Based on (18), update target networks with:
        $\theta^{\mu'} \leftarrow \omega\theta^{\mu} + (1-\omega)\theta^{\mu'}$
        $\theta^{Q'} \leftarrow \omega\theta^{Q} + (1-\omega)\theta^{Q'}$
    end for
end for

the parameters of the target networks are updated based on (18).

V. Numerical Results

In this section, we use Python and TensorFlow to present extensive simulation results of the proposed DDPG-based vehicular edge caching and content delivery algorithm. The implementation is based on the open-source package DDPG [16].

A. Simulation Setup

We consider a network with 4 square areas and each one of them is 500m $\times$500m square area. The network consists of one MBS and 4 RSUs where the MBS is located at the centre of the network. There are 20 vehicles moving along the roads with $v = 60$ km/hr. The density of intersections $\delta_{int} = 2 \times 10^{-3}$ m^{-1}. The driving direction of each vehicle is randomly taken from $\{north, south, west, east\}$. P_{wait} and T_{wait} are set as 0.8 and 2s, respectively. There are 50 contents in the system. The size and the maximal content delivery latency of each content are within the range of $[50, 200]$ MB and $[1, 6]$s, respectively. The caching capabilities of RSUs and vehicles are 10 GB and 5 GB, respectively. The maximal distance of V2V communication is $\gamma = 100$ m. The transmission power of vehicles, RSUs and the MBS for content delivery is 24 dBm, 36 dBm and 43 dBm, respectively. The bandwidth of V2V, V2R, and V2I links are 5 MHz, 10 MHz and 20 MHz, respectively. There are 10 orthogonal sub-carriers. The noise power is $\sigma^2 = 10^{-11}$ mW. We deploy two hidden layers with 100 neurons, and use the activation function of Tanh. The size of mini-batch is set up as 64. The maximum episode is 5000 and the maximum number of steps in each episode is set 25. All learning rates of actor and critic neural network are set as 10^{-3}. The penalty is 100.

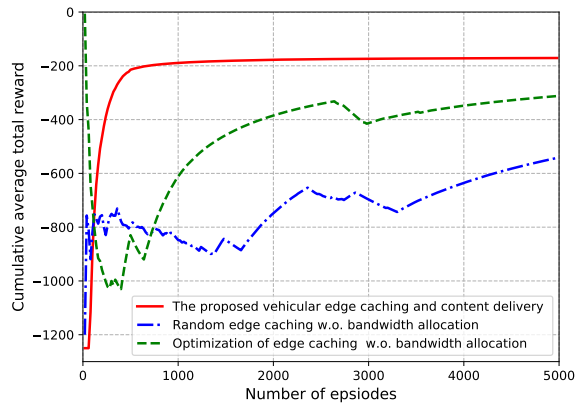

Fig. 2: Cumulative average total reward v.s. number of episodes under different schemes.

To verify the performance of our proposed edge caching and content delivery algorithm, we introduce the following two benchmark schemes,

- *Random edge caching and content delivery w.o. bandwidth allocation*: The strategy of edge caching and content delivery are randomly made without bandwidth optimization.
- *Optimization of edge caching and content delivery w.o. bandwidth allocation*: This scheme optimizes edge caching and content delivery concurrently based on the DDPG algorithm but without bandwidth optimization.

B. Performance Analysis

Fig. 2 shows the comparison of cumulative average total reward with respect to the number of episodes under different schemes. We can draw several observations from Fig. 3. First, the total reward of the proposed algorithm is the highest. The reason is that, in the proposed DDPG-based vehicular edge caching and content delivery scheme, edge caching and content delivery strategy, and bandwidth of the RSUs are jointly optimized while optimization of edge caching and content delivery w.o. bandwidth only considers edge caching and content delivery. The performance of the random edge caching and content delivery scheme is the worst because there is no optimization on edge caching, content delivery and bandwidth allocation. Second, we observe that the total reward of the proposed scheme has a good convergence and it converges after running 1000 episodes. Moreover, all values of cumulative average total rewards are negative. This is

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 16:18:15 UTC from IEEE Xplore. Restrictions apply.

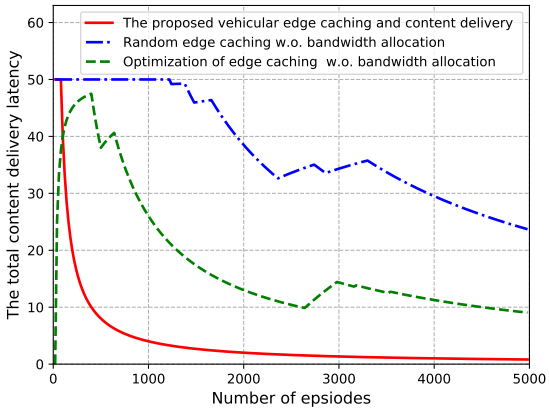

Fig. 3: Content delivery latency v.s. number of episodes under different schemes.

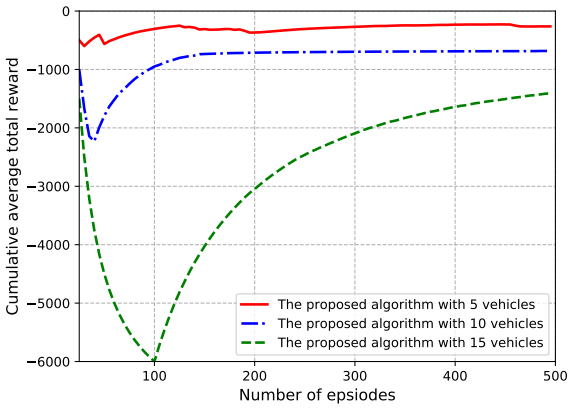

Fig. 4: Cumulative average total reward v.s. number of episodes with different vehicles.

reasonable because to minimize the content delivery latency, we define the immediate reward as a negative value.

Fig. 3 depicts the total content delivery latency under different schemes. We can see that the total content delivery latency of the proposed scheme is obviously lower than the total content delivery latency of two benchmark schemes. It indicates that, in a multi-access vehicular network, 1) better edge caching strategy leads to faster content delivery; 2) optimizing bandwidth of RSUs reduces content delivery latency because of the improvement of data transmission rate.

Fig. 4 shows the comparison of cumulative average total reward with different vehicles. We observe that with the increasing number of vehicles, the cumulative total reward decreases. This is because, more vehicles means more frequent content requests which leads to a higher total content delivery latency. Further, we can see that the fewer the number of vehicles, the faster the DDPG-based algorithm converges.

VI. Conclusions

In this paper, we proposed a multi-access edge caching and content delivery approach for minimizing the total content delivery in vehicular networks. We first presented a multi-access edge caching framework in which RSUs and vehicles with limited caching resources cache some popular contents for content delivery, and vehicles obtain the requested contents via V2V, V2R or V2I links. Then, we formulated a vehicular edge caching and content delivery problem as an MDP and utilized deep reinforcement learning to solve the formulated problem. Specifically, the vehicular edge caching and content delivery algorithm is based on the advanced Deep Deterministic Policy Gradient (DDPG) algorithm. Numerical results demonstrated that the proposed DDPG-based vehicular edge caching and content delivery algorithm significantly outperforms the benchmark policies in terms of content delivery latency.

References

[1] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, pp. 1–1, 2019.

[2] Y. Zhang, R. Yu, S. Xie, W. Yao, Y. Xiao, and M. Guizani, "Home m2m networks: architectures, standards, and qos improvement," *IEEE Commun. Mag.*, vol. 49, no. 4, pp. 44–52, 2011.

[3] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint offloading and resource allocation in vehicular edge computing and networks," in *IEEE GLOBECOM*. IEEE, 2018, pp. 1–7.

[4] K. Zhang, S. Leng, Y. He, S. Maharjan, and Y. Zhang, "Cooperative content caching in 5g networks with mobile edge computing," *IEEE Wirel. Commun.*, vol. 25, no. 3, pp. 80–87, 2018.

[5] X. Huang, Y. Lu, D. Li, and M. Ma, "A novel mechanism for fast detection of transformed data leakage," *IEEE Access*, vol. 6, pp. 35 926–35 936, 2018.

[6] Y. Dai, D. Xu, S. Maharjan, G. Qiao, and Y. Zhang, "Artificial intelligence empowered edge computing and caching for internet of vehicles," *IEEE Wirel. Commun.,to be published*, 2018.

[7] K. Shanmugam, N. Golrezaei, A. G. Dimakis, A. F. Molisch, and G. Caire, "Femtocaching: Wireless content delivery through distributed caching helpers," *IEEE Trans. Inf. Theory*, vol. 59, no. 12, pp. 8402–8413, 2013.

[8] M. Ji, G. Caire, and A. F. Molisch, "Wireless device-to-device caching networks: Basic principles and system performance," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 1, pp. 176–189, Jan 2016.

[9] M. Chen, Y. Hao, L. Hu, K. Huang, and V. K. Lau, "Green and mobility-aware caching in 5g networks," *IEEE Trans. Wirel. Commun.*, vol. 16, no. 12, pp. 8347–8361, 2017.

[10] Y. Dai, D. Xu, S. Maharjan, Z. Chen, Q. He, and Y. Zhang, "Blockchain and deep reinforcement learning empowered intelligent 5g beyond," *IEEE Netw.*, vol. 33, no. 3, pp. 10–17, May 2019.

[11] J. Song, M. Sheng, T. Q. Quek, C. Xu, and X. Wang, "Learning-based content caching and sharing for wireless networks," *IEEE Trans. Commun.*, vol. 65, no. 10, pp. 4309–4324, 2017.

[12] Y. Wei, F. R. Yu, M. Song, and Z. Han, "Joint optimization of caching, computing, and radio resources for fog-enabled iot using natural actor-critic deep reinforcement learning," *IEEE Internet Things J.*, 2018.

[13] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint computation offloading and user association in multi-task mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12 313–12 325, Dec 2018.

[14] M. Li, P. Si, and Y. Zhang, "Delay-tolerant data traffic to software-defined vehicular networks with mobile edge computing in smart city," *IEEE Trans. Veh. Technol.*, vol. 67, no. 10, pp. 9073–9086, 2018.

[15] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[16] https://github.com/floodsung/DDPG.

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 16:18:15 UTC from IEEE Xplore. Restrictions apply.